

A PRACTICAL GUIDE

Building Skills for Claude Code

A step-by-step guide to teaching Claude your workflows, from your first SKILL.md to a tested, shareable skill library.

Based on Anthropic's guide "How to create Skills", expanded for Claude Code.

Contents

1. What a skill is, and why it beats re-explaining yourself
 2. Before you write anything: pick the right task
 3. Step 1 — Define the requirements
 4. Step 2 — Name your skill
 5. Step 3 — Write the description (the part that matters most)
 6. Step 4 — Write the instructions
 7. Step 5 — Install the skill in Claude Code
 8. Step 6 — Test it: normal, edge and out-of-scope
 9. Scaling up: progressive disclosure and bundled files
 10. Limitations and gotchas
 11. Three real-world skills, dissected
 12. Sharing skills with a team
 13. Troubleshooting and FAQ
 14. Quick-reference checklist and templates
- A. Resources

A note on sources

The structure, examples and best practices in this book come from Anthropic's blog post *How to create Skills: key steps, limitations, and examples*. Material marked “*Claude Code specifics*” adds practical detail about working in Claude Code; check the official Claude Code documentation for the latest behaviour, as features evolve quickly.

What a skill is

A **skill** is a folder containing a file named `SKILL.md`: a set of specialised instructions that extends what Claude can do for a specific task or domain. When your request matches the skill, Claude pulls the skill in and follows it. When it doesn't, the skill stays out of the way.

Think of it as writing an onboarding document for a very capable new colleague, once, instead of repeating the same briefing at the start of every conversation.

What skills give you

- **Institutional knowledge, encoded.** Your conventions, formats and house style live in one place.
- **Standardised outputs.** The tenth report looks like the first.
- **Multi-step workflows without repetition.** You describe the process once; Claude runs it on demand.

Skills vs. a pasted prompt

	Pasted prompt	Skill
Reuse	Copy and paste each time	Loaded automatically when relevant
Size	Eats context every time	Description is always visible; the body loads only when needed
Resources	Text only	Can reference scripts, templates and reference files
Sharing	Ad hoc	A folder you can version and distribute

The same skill format works across Claude's surfaces: the Claude apps, Claude Code, and the Claude Developer Platform (API). This book focuses on Claude Code.

Before you write anything

Only build skills for repeated work

The guide's rule of thumb is a simple test: “*Have I done this five times? Will I do it ten more times?*” If both answers are yes, a skill will pay for itself. If it's a one-off, just prompt.

Good candidates

- Applying a brand or style guide to outputs
- Producing a specific file type the same way every time (reports, spreadsheets, decks)
- Code review against your team's checklist
- A release, deploy or incident procedure
- Domain workflows such as SEO optimisation for blog posts

Poor candidates

- One-time tasks
- Tasks too vague to describe a good result for
- Knowledge Claude already has and you don't need to constrain

Pick the right granularity

Single-purpose skills work best. “*SEO optimisation for blog posts*” is a good size: specific enough to trigger reliably, general enough to reuse. “Everything

about marketing” is too broad; “fix the typo in last Tuesday's post” is too narrow.

Shortcut: the skill-creator skill

Anthropic publishes a `skill-creator` skill (in the `anthropics/skills` repository and in Claude.ai) that interviews you with clarifying questions and drafts a skill for you. It's a fast way to get a first draft. You'll still want to read this book to review and improve what it produces.

STEP 1

Define the requirements

Start with the problem, not the file. State what the skill accomplishes with a measurable outcome. Specific beats vague:

✗ “Help with PDFs.”

✓ “Extract financial data from PDFs and format it as CSV.”

Four questions to answer first

1. **What specific task does this accomplish?**
2. **What should trigger it?** Which phrases, file types or situations?
3. **What does success look like?** Required sections, formats, validation checks, quality thresholds.
4. **What edge cases exist?** Missing data, odd formats, ambiguous requests.

Define success criteria explicitly

Tell Claude what a good output looks like. Include required sections, formatting standards, validation checks and quality thresholds. Without them, Claude has to guess where “done” is.

Worked example: a weekly-report skill

Task: Turn raw weekly notes into a status report for leadership.

Triggers: "weekly report", "status update", "summarise my week"

Success: One page. Sections: Wins, Risks, Next week.
Every risk has an owner. No jargon.

Edge: Notes with no risks; notes with duplicate items; notes in bullet fragments.

Keep this note. In the next steps it becomes the name, description and body of the skill.

STEP 2

Name your skill

The name is a clear identifier. Follow three rules:

- Use **lowercase letters with hyphens**.
- Keep it **short**.
- Make it **descriptive** of the job.

Good	Avoid
pdf-editor	PDF Helper Tool v2
brand-guidelines	stuff
weekly-report	Marketing_Everything

Claude Code specifics

In Claude Code the skill's folder name and the `name` field normally match, and the name is also how you can invoke a skill explicitly, by typing `/skill-name`. A short, memorable name is therefore a usability feature, not only a label.

STEP 3

Write the description

The description is **the most critical component of a skill**, because it determines when the skill activates. Claude keeps every skill's name and description in view and decides, from the meaning of your request, whether any skill applies. If the description is weak, the skill never fires, no matter how good the instructions below it are.

Weak vs. strong

Weak: “This skill helps with PDFs and documents.”

Strong: “Comprehensive PDF manipulation toolkit for extracting text and tables, creating new PDFs, merging/splitting documents, and handling forms. When Claude needs to fill in a PDF form or programmatically process, generate, or analyze PDF documents at scale.”

What strong descriptions contain

- **Specific verbs:** extract, create, merge, split, fill.
- **Concrete use cases and file types:** “.docx files”, “PDF forms”, “blog posts”.
- **Clear boundaries:** what the skill is for, and implicitly what it is not for.
- **A “when” clause:** “Use when the user asks to...”, “When Claude needs to...”.

A formula you can reuse

[What it does, with 2-4 specific verbs]. [Outputs or file types].

Use when [situations / phrases / file types that should trigger it].

Not for [nearby tasks it should ignore].

Example

Turns raw weekly notes into a one-page leadership status report with Wins, Risks and Next-week sections. Use when the user asks for a weekly report, status update or summary of their week. Not for daily stand-up notes or long-form project retrospectives.

Semantic matching, not keywords

Claude evaluates descriptions by *meaning*, not by keyword matching. You don't need to stuff every synonym in, but vague wording lowers triggering accuracy. Focus on capabilities and scenarios.

STEP 4

Write the instructions

Below the frontmatter (the name and description) is the body of `SKILL.md`. This is loaded only when the skill is triggered, so it can be detailed. Make it easy to scan and hard to misread.

Structure that works

- **Markdown headers** for hierarchy.
- **Bullet points** for options and rules.
- **Code blocks** for exact examples and commands.
- **Clear sections:** overview, prerequisites, execution steps, examples, error handling, limitations.

Key principle: discrete phases

Break complex workflows into discrete phases, each with clear **inputs and outputs**. “Phase 2 takes the cleaned list from Phase 1 and produces a draft” is far more reliable than a paragraph of prose.

A complete example

```
---
name: weekly-report
description: Turns raw weekly notes into a one-page
leadership
  status report with Wins, Risks and Next-week sections. Use
when
  the user asks for a weekly report, status update or summary
of
  their week. Not for daily stand-ups or long retrospectives.
---

# Weekly Report

## Overview
Convert unstructured notes into a one-page status report that
a
busy executive can read in under two minutes.

## Inputs
- Raw notes (pasted text or a file path)
- Optional: the reporting period

## Steps
1. **Collect.** Read all the notes. List every distinct item.
2. **Classify.** Tag each item as Win, Risk or Next-week.
3. **De-duplicate.** Merge items that describe the same
thing.
4. **Draft.** Write the report using the template below.
5. **Check.** Run the quality checklist; fix anything that
fails.

## Template
**Wins** - up to 5 bullets, outcome first
**Risks** - each with an owner and a mitigation
**Next week** - top 3 priorities

## Quality checklist
- [ ] Fits on one page
- [ ] Every risk has an owner
- [ ] No unexplained acronyms

## Edge cases
```

- No risks in the notes: write "None identified", don't invent any.
- Notes are fragments: ask one clarifying question if the meaning is unclear, otherwise proceed.

Limitations

Does not pull data from other systems; works only from the notes provided.

Writing tips

- Give Claude information it *doesn't already have*: your formats, your thresholds, your names. Don't explain things it knows.
- Show a good and a bad example when quality is subjective.
- Tell it what to do when something is missing or ambiguous.
- Keep the file lean. If it's getting long, split it (see Chapter 9).

STEP 5

Install the skill in Claude Code

In Claude Code, a skill is a directory containing `SKILL.md`. Create the directory structure and the file is picked up.

```
my-project/  
└─ skills/  
    └─ my-skill/  
        └─ SKILL.md
```

Claude Code specifics — where skills live

In practice Claude Code discovers skills from a `.claude/skills/` directory. Two common locations:

- **Personal:** `~/.claude/skills/<skill-name>/SKILL.md` — available in every project.
- **Project:** `.claude/skills/<skill-name>/SKILL.md` inside a repository — commit it so your whole team gets it.

Skills can also arrive through **plugins and plugin marketplaces**. Confirm the exact paths for your version in the Claude Code docs.

Step by step

1. Create the folder: `mkdir -p .claude/skills/weekly-report`
2. Create `SKILL.md` inside it with the frontmatter and body from the previous chapter.
3. Start (or restart) Claude Code in the project so it loads the skill.

4. Ask for something that matches the description, e.g. “Turn these notes into my weekly report”, and watch for Claude to use the skill.

Other surfaces (for reference)

Claude apps (Claude.ai). Add a custom skill in Settings. This requires a Pro, Max, Team or Enterprise plan with code execution enabled. Skills added there are individual to each user, not organisation-wide.

Claude Developer Platform (API). Upload skills through the Skills endpoint:

```
curl -X POST "https://api.anthropic.com/v1/skills" \  
-H "x-api-key: $ANTHROPIC_API_KEY" \  
-H "anthropic-version: 2023-06-01" \  
-H "anthropic-beta: skills-2025-10-02" \  
-F "display_title=My Skill Name" \  
-F "files[]=@my-skill/SKILL.md;filename=my-skill/SKILL.md"
```

The API call and beta header are as published in Anthropic's guide; check the current API docs before relying on them.

STEP 6

Test it

A skill is software for a language model, and it needs tests. Test **triggering** and **execution** separately: a skill can fail to activate, or activate and do the wrong thing, and the fixes are different.

The three-scenario test matrix

Scenario	What you try	What you want
Normal operations	Typical requests the skill should handle perfectly	Skill fires; output meets every success criterion
Edge cases	Incomplete, unusual or messy input; missing data; unexpected formats	Graceful handling per your edge-case section; no invented data
Out-of-scope requests	Unrelated tasks, and tasks near the skill's territory	Skill stays dormant

Triggering tests

- Does it activate when you phrase the request in several different ways?
- Does it stay inactive when the request is irrelevant?

Functional tests

- Is the output consistent across repeated runs?
- Is it usable without manual fixing?
- Does the documentation inside the skill match what actually happens?

Build a test-case library

Keep a short list of prompts, with expected behaviour, next to the skill. Re-run it every time you edit the skill.

```
weekly-report / test cases
1. [normal]   "Make my weekly report from these notes: ..."
-> fires
2. [normal]   "Status update for leadership, notes below"
-> fires
3. [edge]     Notes with zero risks
-> "None identified"
4. [edge]     Notes as 3-word fragments
-> one clarifying question
5. [out]      "Write my daily stand-up"
-> does NOT fire
6. [out]      "Summarise this research paper"
-> does NOT fire
```

Iterate

- **Doesn't activate?** Broaden or sharpen the description: add verbs, file types, scenarios.
- **Activates but results vary?** Add specificity to the instructions: templates, examples, checklists.
- **Activates when it shouldn't?** Add a “Not for...” clause to the description.

Scaling up: progressive disclosure

Once a skill handles several processes, one giant `SKILL.md` becomes slow and confusing. Use a **menu approach**: keep `SKILL.md` short, describe the available options, and point to separate files for each distinct process. Claude loads only the file that is relevant.

```
report-suite/
├── SKILL.md           # overview + decision tree (short)
├── weekly.md          # weekly report process
├── monthly.md         # monthly report process
├── templates/
│   └── executive-summary.md
└── scripts/
    └── validate.py    # deterministic checks
```

Example SKILL.md body

```
# Report Suite

Choose the process that matches the request, then read that
file:

- Weekly status report -> read weekly.md
- Monthly business review -> read monthly.md

Always finish by running scripts/validate.py on the output.
```

Why this works

- **Context stays lean.** Unused processes cost nothing.

- **Maintenance is easier.** Edit one process without touching the others.
- **Scripts make checks deterministic.** Use code for things code does well, such as validation, and language for judgment.

The official `docx` skill is a good model: a lean main file with a decision tree that routes Claude to the right workflow.

Limitations and gotchas

Triggering is probabilistic and semantic

- Claude matches descriptions by meaning, not by exact keywords, so a vague description reduces accuracy.
- **Multiple skills can activate at once** for a complex request. Design skills so they cooperate rather than collide.

Size matters

A single bloated skill is harder for Claude to follow and costs more context. Split multi-process skills with the menu approach from Chapter 9.

Availability varies by surface

- In the Claude apps, custom skills need a paid plan with code execution enabled, and are **per user**, not organisation-wide (organisation-wide management was described as coming).
- Skills in Claude Code are file-based, so they can be shared through version control or plugins.

A skill is guidance, not a guarantee

Instructions steer Claude; they don't hard-enforce behaviour. For anything that must be exactly right, bundle a script that checks the result, and review outputs while the skill is new.

Security

Skills can include instructions and executable scripts. Treat a skill like software you install: only use skills from sources you trust, and read third-party skills before enabling them.

Three real-world skills, dissected

These examples come from Anthropic's published skills. Study how each balances a precise description, a lean body and concrete detail.

1. `docx` — document creation and editing

Description (abridged): “Comprehensive document creation, editing, and analysis with support for tracked changes, comments, formatting preservation, and text extraction. When Claude needs to work with professional documents (.docx files) for: (1) Creating new documents, (2) Modifying or editing content, (3) Working with tracked changes, (4) Adding comments...”

What the instructions cover:

- Reading and analysing content (text extraction with Pandoc; raw XML access)
- Creating new Word documents (with the docx-js library)
- Editing existing documents (a Python Document library)
- A redlining workflow for document review
- Converting documents to images (DOCX → PDF → JPEG)

Why it works: a clear decision tree routes Claude to the right workflow; progressive disclosure keeps the main file lean; it includes good and bad implementation examples. The description enumerates numbered use cases, which makes triggering reliable.

2. brand-guidelines — applying a visual identity

Description: “Applies Anthropic’s official brand colors and typography to any sort of artifact that may benefit from having Anthropic’s look-and-feel.”

Technical details inside:

Item	Value
Main colours	Dark #141413 · Light #faf9f5 · Mid gray #b0aea5 · Light gray #e8e6dc
Accent colours	Orange #d97757 · Blue #6a9bcc · Green #788c5d
Typography	Poppins for headings · Lora for body text
Also covered	Font management and fallbacks; applying colours via RGB values

Why it works: it gives Claude precise, actionable information it doesn't inherently possess, and the description is a clear trigger. This is the archetype of “encode what only you know”.

3. frontend-design — creative work with guardrails

Description: “Create distinctive, production-grade frontend interfaces with high design quality. Use this skill when the user asks to build web components, pages, or applications.”

Design thinking it requires: understand the purpose and audience; pick a bold aesthetic direction (brutally minimal, maximalist, retro-futuristic, organic, luxury, playful, editorial, brutalist, art deco, soft, industrial...); commit to intentional execution.

Aesthetic guidelines:

- **Typography:** distinctive fonts; pair display fonts with refined body fonts.
- **Colour and theme:** a cohesive aesthetic with CSS variables; dominant colours with sharp accents.
- **Motion:** prefer CSS-only; focus on high-impact moments like page load.
- **Spatial composition:** asymmetry, overlap, diagonal flow, unexpected layouts.
- **Backgrounds and detail:** atmosphere and depth via gradients, noise, patterns, transparency, shadows.

What it tells Claude to avoid: generic aesthetics such as overused fonts (Inter, Roboto, Arial), clichéd colour schemes, predictable layouts and cookie-cutter design.

Why it works: it leaves room for creativity while setting clear boundaries, quality standards and technical scaffolding. Note the explicit “avoid” list: negative examples are as useful as positive ones.

Sharing skills with a team

By team size

Team	Approach
Small	Use a shared template with name, description, instructions and version info.
Medium / large	Establish governance: designated skill owners, a central wiki or shared drive, usage examples, versioning with a changelog, and quarterly reviews.
All sizes	Document the business purpose, assign clear ownership, create onboarding materials, and track the value the skill delivers.

Claude Code specifics

The simplest way to share is to commit `.claude/skills/<name>/` to your repository: skills then follow the code, get reviewed in pull requests and are versioned by git. For wider distribution, package skills in a plugin and publish it to a marketplace your team can install from.

A skill's metadata template

```
Name:          weekly-report
Owner:         jane.doe
Purpose:       Standardise leadership status updates
Version:       1.2.0
Last reviewed: 2026-09-30
Changelog:     1.2.0 - added "None identified" rule for
empty risks
Test cases:    see tests.md
```

Troubleshooting and FAQ

How do I write descriptions that actually trigger?

Focus on capabilities and scenarios. Include action verbs, specific file types and clear use cases.

How does Claude decide which skills to invoke?

By semantic understanding of the request and each skill's description, not keyword matching. Multiple skills can activate together for complex requests.

What's the right granularity?

Single-purpose. “SEO optimisation for blog posts” balances specificity with reusability.

How do I debug a skill?

Separate the two failure modes. If it doesn't activate, broaden or sharpen the description. If it activates but results vary, add specificity to the instructions. Maintain a test-case library and re-run it after each change.

The skill fires too often.

Narrow the description and add a “Not for...” clause. Check whether another skill overlaps.

The output ignores part of my instructions.

The body may be too long or buried. Move critical rules to the top, use a checklist, and split rarely used material into separate files.

Do skills work across Claude Code, the apps and the API?

The skill format is shared, but each surface installs skills differently, and availability and limits vary. See Chapter 7.

Quick-reference checklist

The six steps

1. **Define** the task, triggers, success criteria and edge cases.
2. **Name** it: lowercase, hyphenated, short, descriptive.
3. **Describe** it: specific verbs, concrete use cases, clear boundaries.
4. **Instruct**: headers, phases with inputs/outputs, examples, error handling.
5. **Install**: `.claude/skills/<name>/SKILL.md`.
6. **Test**: normal, edge, out-of-scope; then iterate.

Pre-flight checklist

- ☐ I've done this task 5+ times and will do it 10+ more
- ☐ The name is lowercase-with-hyphens
- ☐ The description has verbs, use cases and a “when” clause
- ☐ Success criteria are explicit
- ☐ Edge cases and errors are covered
- ☐ Long content is split into separate files
- ☐ I have a test-case library with normal, edge and out-of-scope prompts
- ☐ The skill has an owner and a version

Starter template

```
---
name: your-skill-name
description: [Specific verbs + outputs]. Use when [triggers].
  Not for [exclusions].
---

# Your Skill Name

## Overview
[One paragraph: purpose and outcome]

## Prerequisites
[Inputs, tools, access]

## Steps
1. [Phase 1: input -> output]
2. [Phase 2: input -> output]

## Examples
[One good example, optionally one bad]

## Error handling and edge cases
[What to do when X is missing or Y is odd]

## Limitations
[What this skill does not do]
```

Resources

- **Source article:** claude.com/blog/how-to-create-skills-key-steps-limitations-and-examples
- **Skills repository and skill-creator:** github.com/anthropics/skills (including `skill-creator`)
- **Claude Code documentation:** code.claude.com/docs/en/overview
- **Skills cookbook:** platform.claude.com/cookbook/skills-notebooks-01-skills-introduction
- **API documentation:** docs.anthropic.com
- **Claude Academy:** academy.claude.com

This is an independent study guide summarising and extending Anthropic's public guidance. Product behaviour and file locations may change; the official documentation is authoritative.